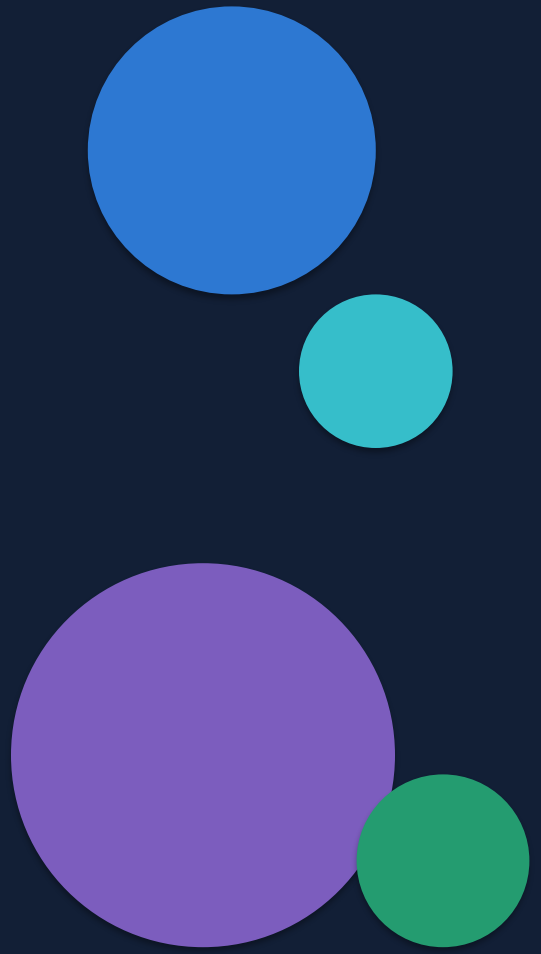
HTTP & Its Methods



How clients and servers communicate on the web

Requests • Responses • Methods • Status Codes • HTTPS

Presented by: Debanshika Das

What we will learn

01 HTTP basics

Meaning of HTTP and client–server communication.

02 Request & response

How messages travel between client and server.

03 HTTP methods

GET, POST, PUT, PATCH and DELETE.

04 Status codes

Success, redirection and error categories.

05 HTTPS & practice

Secure communication and a Node.js API demo.

Goal: understand the concepts, then see them working in code.

What is HTTP?

HTTP

Hypertext Transfer Protocol

HTTP is an application-layer protocol used by clients and servers to exchange messages.

Communication rules

A protocol defines how two systems exchange information.

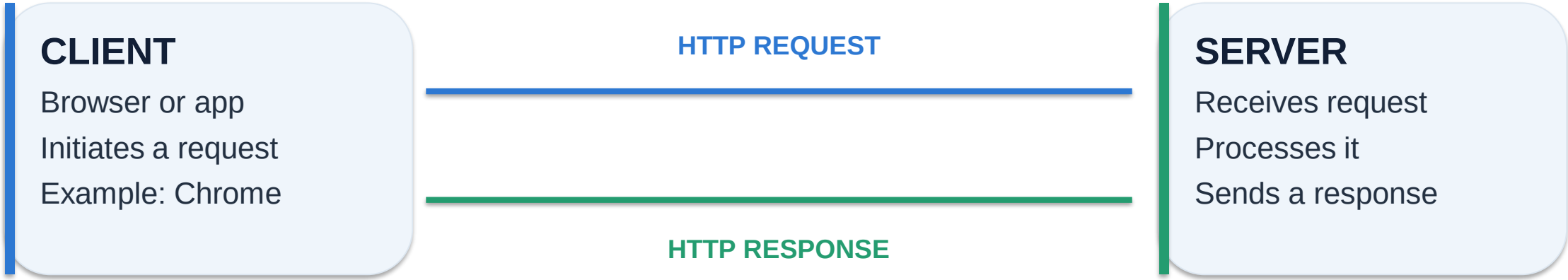
Web communication

Browsers use HTTP to request resources from web servers.

Request–response model

The client sends a request; the server returns a response.

Client–server architecture



Analogy: a customer asks a restaurant for something; the restaurant responds.

The HTTP request–response cycle

1. **Client sends request**
2. **Server receives request**
3. **Server processes request**
4. **Server sends response**
5. **Client displays result**

Example request

GET /students asks for student records.

Example response

The server may return data with 200 OK.

What is inside an HTTP request?

```
GET /students HTTP/1.1
Host: example.com
Accept: application/json
```

Request line

Method + path + HTTP version.

Headers

Metadata such as Host, Accept and Content-Type.

Body (when needed)

Data sent with POST, PUT or PATCH. GET usually has no request body.

Remember

Not every request contains a body.

What is inside an HTTP response?

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id": 1, "name": "Aarav"}
```

Status line

HTTP version + status code.

Headers

Metadata about the response.

Response body

Returned content: JSON, HTML or plain text.

Example

200 OK means the request succeeded.

HTTP methods at a glance

GET	Retrieve a resource
POST	Submit data / create a resource
PUT	Replace a resource
PATCH	Partially modify a resource
DELETE	Delete a resource

The method tells the server what action the client wants to perform.

GET and POST

GET • Retrieve data

- Requests a representation of a resource.
- Example: GET /students
- Used to display a list or view details.
- Safe and idempotent by HTTP semantics.

POST • Submit data

- Sends data to the server for processing.
- Commonly used to create a resource.
- Example: POST /students
- Not generally idempotent.

Example: view student records (GET) | register a student (POST)

PUT, PATCH and DELETE

PUT • Replace

Replaces the target resource's representation with the representation sent by the client.

Example:

PUT /students/1

PATCH • Partial update

Applies partial modifications to a resource.

Example:

PATCH /students/1

Change only the email field.

DELETE • Remove

Requests removal of the target resource.

Example:

DELETE /students/1

PUT usually replaces a full representation; PATCH applies partial changes.

HTTP status code categories

1xx	2xx	3xx	4xx	5xx
Informational	**Success**	**Redirection**	**Client error**	**Server error**
Request received; continuing	Request successfully handled	Further action may be needed	Problem with the request	Server failed to fulfil request

Examples: 200 OK • 201 Created • 301 Moved Permanently • 400 Bad Request • 401 Unauthorized • 404 Not Found • 500 Internal Server Error

HTTP vs HTTPS

HTTP

- HTTP itself does not encrypt communication.
- Data may be exposed if intercepted in transit.
- URL commonly begins with http://

HTTPS

- HTTP communication protected using TLS.
- Provides encryption and server authentication.
- URL commonly begins with https://

HTTPS = HTTP over a connection secured with TLS.

Practical: build a student API

We will create a small API using Node.js's built-in HTTP module.

GET

Read the student list

GET /students

POST

Add a new student

POST /students

PUT / PATCH / DELETE

Update or remove a student

/students/:id

This demo stores records in memory. Data resets when the server restarts.

Server setup: Node.js HTTP module

```
const http = require("node:http");

let students = [
  { id: 1, name: "Aarav", email: "aarav@example.com" },
  { id: 2, name: "Priya", email: "priya@example.com" }
];

const server = http.createServer(async (req, res) => {
  // Handle the incoming request here
});

server.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

http module

Built into Node.js; no extra install needed.

createServer()

Creates a server and handles requests.

listen(3000)

Starts listening on port 3000.

Handling a GET request

```
if (req.method === "GET" && req.url === "/students") {
  res.writeHead(200, {
    "Content-Type": "application/json"
  });

  res.end(JSON.stringify(students));
}
```

Check the method

req.method tells us which method was used.

Set status + headers

writeHead() sets status and response headers.

Send the body

res.end() completes the response.

Show the full implementation and all five routes in VS Code.

Testing the API with curl

```
# GET
curl http://localhost:3000/students

# POST
curl -X POST http://localhost:3000/students \
  -H "Content-Type: application/json" \
  -d '{"name":"Debanshika","email":"deban@example.com"}'

# Then demonstrate PUT, PATCH and DELETE in Terminal.
```

Start the server

node server.js

Send requests

Use a second Terminal tab for curl.

Observe responses

Check status codes and returned JSON.

Quick recap

HTTP

Application-layer protocol for client–server communication.

Request / response

Client asks; server processes and replies.

Methods

GET • POST • PUT • PATCH • DELETE

Status codes

1xx info • 2xx success • 3xx redirect • 4xx client • 5xx server

HTTPS

HTTP communication protected using TLS.

Key idea: HTTP defines how a client asks and how a server responds.

THANK YOU

HTTP & Its Methods

Questions?

Advanced Programming • Video 2